

Los cuatro contratos de esta sección son fijos: rutas, nombres de campos y códigos de respuesta deben coincidir exactamente, porque tu API la va a llamar el código de otra persona.

9.1 Reglas comunes

- Rutas bajo `/api/v1/integracion/**`, protegidas con la cabecera `X-API-KEY` (sin JWT). Clave incorrecta o ausente: 401.
- Cada aplicación arranca en su puerto: CoffeeStation `8081`, FitZone `8082`, CineTown `8083`, ParkControl `8084`. En clase, la URL del compañero será `http://<IP-del-compañero>:<puerto>`.
- URL y API key del servicio remoto se configuran en `application.yml` a partir de variables de entorno (`SAFAREYES_<SERVICIO>_URL`, `SAFAREYES_<SERVICIO>_API_KEY`).
- El cliente usa `RestClient` con tiempo de conexión y lectura de 3 segundos, y trata por separado: respuesta correcta, 4xx (rechazo de negocio), 5xx y tiempo agotado o conexión rechazada (servicio no disponible).
- La operación principal **nunca se bloquea** por un fallo de integración. La respuesta de tu endpoint incluye un bloque `integracion` que informa del resultado.
- Las API solo intercambian los datos imprescindibles (principio de minimización del RGPD): nunca nombres, teléfonos ni datos de pago.

```
JSON
"integracion": {
  "servicio": "FITZONE",
  "estado": "OK",
  "detalle": "Socio activo: descuento del 10 %
aplicado"
}
```

Estado	Significado
OK	La otra API respondió correctamente y se aplicó el resultado
NO_APLICA	No había nada que integrar (p. ej. socio sin matrícula)
RECHAZADO	La otra API respondió 4xx
NO_DISPONIBLE	5xx, tiempo agotado o sin conexión

Contrato	Expone	Consume	Momento de la llamada
A	CoffeeStation	CineTown	Tras confirmar una compra de entradas
B	FitZone	CoffeeStation	Al crear un pedido con <code>dniSocio</code>
C	ParkControl	FitZone	Tras una entrada <code>PERMITIDO</code> de un socio con matrícula
D	CineTown	ParkControl	Al pagar un ticket con <code>codigoCine</code>

9.2 Contratos

A Cupón de cine · expone CoffeeStation

POST /api/v1/integracion/cupones

PETICIÓN

```
{
  "email": "ana.lopez@correo.es",
  "origen": "CINETOWN",
  "referencia": "K7Q2M9XA"
}
```

RESPUESTA 201 CREATED

```
{
  "codigo": "CINE-4F7K2Q",
  "email": "ana.lopez@correo.es",
  "tipo": "PORCENTAJE",
  "valor": 20.00,
  "validoHasta": "2026-10-21",
  "usosMaximos": 1
}
```

- CoffeeStation decide el descuento (20 %, 7 días, 1 uso). El que llama no puede fijarlo.
- **Idempotencia**: si llega otra petición con la misma `referencia`, se devuelve el cupón ya creado con `200 OK`, sin duplicarlo.
- Errores: 400 (email no válido), 401.

B Estado de socio · expone FitZone

GET /api/v1/integracion/socios/{dni}/estado

PETICIÓN

Sin cuerpo. El DNI va en la ruta.

RESPUESTA 200 OK

```
{
  "dni": "12345678Z",
  "socioActivo": true,
  "cuotaAlCorriente": true,
  "plan": "PREMIUM"
}
```

- CoffeeStation aplica el descuento de socio solo si `socioActivo` y `cuotaAlCorriente` son `true`.
- Errores: 400 (DNI con formato no válido), 401, 404 (no es socio).

C Bonificación de gimnasio · expone ParkControl

POST /api/v1/integracion/bonificaciones

PETICIÓN

```
{
  "matricula": "1234BCD",
  "origen": "FITZONE",
  "referencia": "ACC-000123"
}
```

JSON

RESPUESTA 200 OK

```
{
  "matricula": "1234BCD",
  "ticketId": 842,
  "promocion": "GIMNASIO",
  "minutosAplicados": 60,
  "importeActual": 1.35
}
```

JSON

- ParkControl decide los minutos (60) a partir de la promoción **GIMNASIO**. La petición no los incluye.
- Errores: 400, 401, 404 (la matrícula no tiene ticket activo), 409 (la promoción ya se aplicó a ese ticket), 422 (se superarían los 180 minutos de promoción).

D Canje de entrada de cine · expone CineTown

POST /api/v1/integracion/compras/{codigo}/canje-parking

PETICIÓN

```
{
  "matricula": "1234BCD"
}
```

JSON

RESPUESTA 200 OK

```
{
  "codigo": "K7Q2M9XA",
  "valido": true,
  "fechaSesion": "2026-10-14T18:30:00",
  "canjeadoEn": "2026-10-14T21:05:12"
}
```

JSON

- El canje es válido si la compra existe, está **CONFIRMADA**, su sesión es del mismo día y no se ha canjeado antes. CineTown marca la compra como canjeada y guarda la matrícula.
- Errores: 401, 404 (código inexistente), 409 (ya canjeada), 422 (compra cancelada o sesión de otro día).

9.3 Cómo desarrollar sin depender del compañero

1. Implementa y prueba primero el endpoint que expones, con su API key, desde Postman.
2. Para tu cliente, levanta un simulador del servicio remoto que devuelva las respuestas de estos contratos (WireMock o un controlador en un perfil **mock**), incluidos los errores y un retraso de 5 segundos para probar el tiempo de espera.
3. Cuando los dos estéis listos, intercambiad URL y API key y haced la prueba conjunta. Documenta en **docs/integracion.md** las peticiones reales y un caso de fallo (compañero apagado).